

DLCA Lab 4: Simulator

Week 1

August 26, 2025

1 Introduction

In this week's lab, you shall be learning how to use a state-of-the art microarchitectural simulator (Champsim) to see the effects of different microarchitectural policies on different workloads.

2 Necessary tools

We shall be using make and g++ in order to compile C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ make
make: *** No targets specified and no makefile found.  Stop.
$ g++
g++: fatal error: no input files
compilation terminated.
```

You will also need to ensure that your system has the traces installed, this can be checked by running:

```
$ ls $CS231_TRACE_DIR
bwaves.xz gcc.xz mcf.xz sssp.gz
```

You must also ensure that Champsim has been installed and setup in the Desktop folder of your lab machine, there must be a folder on your desktop called Champsim. You can ensure it is correct by running the following from your desktop:

```
ls Champsim/
bin branch btb champsim_config.json CITATION.cff config.sh
inc LICENSE Makefile obj prefetcher replacement run_champsim.sh src storage tracer
```

3 Structure of submission/templates

```
your-roll-no/
|- task1/
   |- pc-based.cc
|- task2/
   |- probabilistic-lru-0.5.cc
```

```
| - pc-based.cc
| - random.cc
| - nru.cc
| - rrip.cc
| - champsim_config.json
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

4 Tasks

This assignment consists of two subtasks. These tasks are expected to be submitted in-lab. The first task is intended to get you comfortable with modifying code in Champsim while the second one is intended for you to make a meaningful modification to a replacement policy. Champsim can be a bit overwhelming to work with, which is why we will be holding your hand through the first task and getting you through it. Our goal as TAs is to ensure every student attending the lab is able to solve task 1.

Some background

How to build Champsim?

In this lab, we want to focus on LLC Cache replacement policies in Champsim. Therefore, we will be building champsim with specific LLC replacement policies. Suppose we want to build champsim with the random replacement policy at the LLC, to do this, you must ensure the following:

1. In the Champsim/replacement/ folder, there must be a folder called random/ with a file called random.cc
2. random.cc must contain the following functions:
 - (a) void CACHE::initialize_replacement
 - (b) uint32_t CACHE::find_victim
 - (c) void CACHE::update_replacement_state
 - (d) void CACHE::replacement_final_stats
3. In the Champsim/ folder, the file champsim_config.json must have "executable_name": "bin/lru" and

```
"LLC": {
    ....
    "replacement": "random"
},
```

4. Run ./config.sh champsim_config.json && make

This builds the champsim binary with the required random replacement policy.

How to run Champsim?

We have given you a tool to run Champsim, the tool takes in two arguments:

1. `TRACE_FILE`: This is the workload which is run on the simulator, you have been given four workloads which are found in the `$CS231_TRACE_DIR` folder
2. `BINARY`: This is the specific champsim binary which contains the simulator on which to run the workload

An example run of champsim looks like:

```
$ bash run_champsim.sh $CS231_TRACE_DIR/sssp.gz random
```

```
*** ChampSim Multicore Out-of-Order Simulator ***
...
```

The replacement policy functions

To implement your replacement policy you will need to understand what the replacement policy functions are:

1. `void CACHE::initialize_replacement`: This function is called before simulation begins. It is there so that you can set up your data structures that you may be using.
2. `uint32_t CACHE::find_victim`: This function is called at the time that an eviction needs to occur from the cache set. You need to return the way number of the cache line to be evicted.
3. `void CACHE::update_replacement_state`: This function is called whenever there is a cache hit to a block and whenever a block is filled into the cache.

Task 1: A naive PC-Based Replacement Policy

Cache lines are inserted into the cache due to specific load or store instructions. Thus, every cache line has an instruction pointer(IP)/program counter(PC) associated with it. The idea behind this replacement policy is that some PCs may be showing large amounts of reuse while others may not be showing much reuse. The PC-Based replacement policy works as follows:

1. For each PC, there is an 8-bit count associated with it, called `pcCount`, set to zero initially
2. Insertion: When a cache line is inserted, it implies that this PC caused a cache miss, thus the `pcCount` is decremented (unless it goes negative, in which case do not decrement)
3. Promotion: When a cache line receives a cache hit, the `pcCount` associated with its PC is incremented
4. Eviction: From among all the cache lines in the set, choose the one with the lowest `pcCount` and evict it

However, there is an issue. PCs are typically 48 bits in Champsim, furthermore, you can't have the counter be incremented indefinitely since we have a finite amount of storage space. Thus, for each PC, we shall be associating the lower 16 bits of the PC with an 8-bit counter. Meaning the maximum value of `pcCount` is 255.

Champsim allows you to access the PC associated with a particular cache line in a particular set, way using `this->block[set * NUM_WAY + way].ip` inside a member function of the cache class (note that all four of the functions are member functions of the cache class).

Task 2: I am speed

Across the two groups and the folks who went home for diwali, we have covered 5 different replacement policies. So far we have been using our replacement policies only for the LLC Cache, in this task, you

are free to edit the replacement policies of the L1D, L1I, L2 and LLC caches. Similar to how you set the replacement policy for the LLC cache, you can set the replacement policy for the other caches. Your goal here is to mix and match, use different replacement policies (from among the ones that will be described in this section) at different cache levels. The statistic for you to maximize is the *geometric mean* of the Cumulative IPCs across the different workloads. You must keep the name of your champsim binary as

```
L1I-<replacement-policy-at-L1I>-L1D-<replacement-policy-at-L1D>-L2-
<replacement-policy-at-L2>-LLC-<replacement-policy-at-llc>
```

for example, if I am using random at all levels, my binary will be

```
L1I-random-L1D-random-L2-random-LLC-random
```

The grading for this task will be done relative to each other. Here are the replacement policies:

1. PC-based:

Cache lines are inserted into the cache due to specific load or store instructions. Thus, every cache line has an instruction pointer(IP)/program counter(PC) associated with it. The idea behind this replacement policy is that some PCs may be showing large amounts of reuse while others may not be showing much reuse. The PC-Based replacement policy works as follows:

- (a) For each PC, there is an 8-bit count associated with it, called pcCount, set to zero initially
- (b) Insertion: When a cache line is inserted, it implies that this PC caused a cache miss, thus the pcCount is decremented (unless it goes negative, in which case do not decrement)
- (c) Promotion: When a cache line receives a cache hit, the pcCount associated with it's PC is incremented
- (d) Eviction: From among all the cache lines in the set, choose the one with the lowest pcCount and evict it

2. random:

A random replacement policy is one in which the candidate cache line to be evicted is selected at random. There is no promotion or insertion policy since all lines have an equal probability of being evicted.

3. probabilistic-lru-0.5:

Same eviction and promotion policy as LRU, to recap, the policies for LRU are:

- (a) Eviction policy: Evict the cache line which was least recently accessed from among the cache lines in the set
- (b) Insertion policy: Insert a cache line as having been most recently accessed
- (c) Promotion policy: When a cache line receives a hit, promote it to having been most recently accessed

We will be making a modification in the insertion policy, we shall be inserting lines as having been most recently accessed with a probability of 0.5 and least recently accessed with a probability of 0.5. We will sample a random number modulo 100, divide it by 100 and if it is less than 0.5, we insert the cache line in the least recently accessed position. Otherwise, we insert in most recently accessed position.

4. not-recently-used:

The Not-Recently-Used NRU policy requires that every cache line has a bit associated with it, called as the nruBit. If the bit is set to 1, it indicates that the cache line was recently used, if it is set to 0, it indicates that the line was not recently used. The NRU policy has the following components:

- (a) Insertion: When a cache line is inserted, set the nruBit of the cache line to 1

- (b) Promotion: When a cache line receives a hit, set the `nruBit` of the cache line to 1
- (c) Eviction: Evict a random line from among all the lines having the `nruBit` 0, if all lines in that set have `nruBit` 1, then set all the lines' `nruBit` to 0 and evict randomly from among all the lines

5. re-reference interval prediction:

The Re-Reference Interval Prediction RRIP policy requires that every cache line has 3 bits associated with it, called as the `rripBits` which have a value called the `rripVal`. The replacement policy works as follows:

- (a) Insertion Policy: Insert with the `rripVal` set to 1
- (b) Promotion Policy: On a hit, set the `rripVal` set to 7
- (c) Eviction Policy: Evict randomly from among the cache ways having 0 as the `rripVal`. If there are no such lines, proceed as follows:
 - i. Suppose the lowest `rripVal` among the cache lines in the set is x
 - ii. For each cache line, if it had an `rripVal` of y , change it's `rripVal` to $y - x$
 - iii. Evict randomly from among the cache ways having 0 as the `rripVal`